Fine-tuning a Lean-4-only GPT: does RLHF beat supervised fine-tuning?

nanoGPT (0.8M parameters) · pretraining, SFT, RLHF and alternatives, measured with a proof checker and Lean 4

Status (4 Nov 2026). The first full run is complete (section 4.1). A larger replication, with three seeds, two more post-training arms and every answer compiled by Lean 4, is running now; its numbers so far are in section 4.2. This report is updated in place at the same link as the replication finishes.

Summary

The model. A 0.8M-parameter GPT (the nanoGPT architecture) that I pretrained, fine-tuned and post-trained from scratch on nothing but Lean 4: propositional and first-order proofs, `Nat` arithmetic, and linear equations over `Int`. There is no natural language and no safety training anywhere in the pipeline.

What I changed: the post-training method. From one supervised fine-tuned (SFT) checkpoint I compared RLHF, which trains a Bradley-Terry reward model on checker-labelled comparisons of the model's own answers and then runs PPO against it with a KL penalty, with PPO run directly against the proof checker. The replication adds rejection-sampling fine-tuning (keep the sampled proofs that check, fine-tune on them) and a "more SFT" control. All arms that sample get the same sample budget.

How I measured it. Greedy accuracy on held-out problems. The train/test split hashes a canonical form of each problem, so renamed or respelled copies of test problems never appear in training. Answers are judged by checkers that are fuzzed against real Lean 4. In the replication, every answer is also compiled by Lean, results are compared with paired bootstrap confidence intervals, and each arm is run with three seeds. I also tracked the reward model's accuracy on held-out comparisons, and the reward against true accuracy during PPO.

What I learned.

- RLHF did not beat SFT: 38.7% against 38.0% on 1,200 test problems, inside the ±1.4-point standard error.
- The reward model was the bottleneck. It ranked only 57% of held-out pairs correctly (55.8% in the replication) while fitting its training pairs.
- With a looser KL penalty, PPO exploited the reward model: the reward rose while checked accuracy fell from 41% to 8%.
- A PPO iterate that looked 4.6 points better on the validation split was the best of 16 noisy evaluations, and did not hold up on the test split.
- PPO against the checker itself was no better at this scale. RL mostly redistributed probability among answers the SFT model could already produce.

1. The model and the data

A problem is a Lean 4 declaration cut in two: the model sees the header and writes the rest, and the whole declaration must compile.

domain	the model sees	and writes
prop	`example (p q s : Prop) : p ∧ (¬p ∧ ¬q) ∧ s → ¬p :=`	`fun h0 => h0.2.1.1`
fol	`example (α : Type) (P R : α → Prop) (q : Prop) : (∃ x, P x) → (∀ x, P x → q ∧ R x) → ∃ x, q ∧ R x :=`	`fun ⟨x, h0⟩ h1 => ⟨x, h1 x h0⟩`
nat	`example : (10 - 10 * 6) % 15 + 7 =`	`7 := by decide`
linear	`example (x : Int) (h : 7 * x - 14 = x + 10) : x =`	`4 := by omega`

- **Model.** A character-level GPT-2-style transformer with 4 layers, 4 heads, width 128, a 256-token context, tied embeddings and 803,072 parameters.
- **Tokens.** Sequences are `<bos> header <ans> answer <eos>`. Pretraining documents are whole declarations.
- **Checkers.** Each domain has a generator and a checker: a Python implementation of the fragment of Lean 4 the domain uses, fast enough to judge every sample during training. A checker must be sound with respect to Lean: it may reject a proof Lean accepts, never the reverse.
- **Testing the checkers.** The tests compile generated declarations, and hundreds of thousands of mutated answers the checkers accept, with Lean 4.34. A JavaScript port of every checker drives an in-browser chat page, and the tests check that it agrees with the Python one.

2. The training pipeline

stage	data	objective
pretraining	160k whole declarations (train split)	next-token cross-entropy
SFT	80k (header, shortest reference proof) pairs	cross-entropy on the answer only
reward model	6,000 prompts × 8 SFT samples, pairs ranked by the checker (a correct proof beats an incorrect one; between two correct proofs, the shorter wins)	Bradley-Terry, −log σ(r_chosen − r_rejected)
RLHF	train-split headers	clipped PPO on RM score − β·KL(π ‖ π_SFT), GAE, critic initialised from the RM
PPO vs checker	the same	clipped PPO on ±1 from the checker − β·KL
RFT (replication)	the policy's own samples that check	cross-entropy, a few epochs per round
more SFT (replication, control)	fresh reference proofs	cross-entropy

The checker labels the comparisons in place of human raters. This makes them noise-free, so any gap between the reward model and the checker is the reward model's own error. PPO optimises the learned reward model, not the checker, exactly as RLHF with human labels does.

3. How it is measured

- **Held-out problems.** Problems are split train/val/test (90/5/5) by a hash of their canonical form. For logic, atoms are renamed in order of appearance and bound variables normalised. For arithmetic, commuted

operands and swapped sides of an equation are identified. A respelled test problem therefore cannot leak into training.

- **Metric.** Greedy pass@1. An answer counts only if it is finished and accepted.
  - First run: judged by the checkers.
  - Replication: also compiled with Lean 4. Checker false positives and false negatives are counted, and every model is scored on the same 2,000 test problems (500 per domain).
- **Statistics** (replication). Wilson intervals for each accuracy. Paired bootstrap intervals and exact McNemar tests for each difference from SFT. Three training seeds per arm, plus sampled pass@k (16 samples) to separate gains in reliability from gains in what the model can produce at all.
- **Diagnostics.**
  - The reward model's pairwise accuracy and AUC on prompts it was not trained on.
  - The reward model's score against checked accuracy across PPO iterations, to detect reward hacking.

4. Results

4.1 First run (complete)

Accuracy on 1,200 held-out test problems (300 per domain), greedy decoding, judged by the checkers:

checkpoint	prop	fol	nat	linear	all
pretrained (completing the declaration as a document)	29.3	54.0	16.0	8.0	26.8
SFT	47.0	67.7	21.7	15.7	38.0
RLHF (PPO against the reward model; iterate chosen on val, 90 of 150)	46.7	67.0	21.0	20.0	**38.7**
RLHF, last iterate	44.7	65.3	20.0	17.0	36.7
PPO against the checker itself (iterate 100)	47.0	67.3	24.0	14.3	38.2

- **Pretraining and SFT do the work.** The base model already proves a quarter of the problems by completing them as documents; SFT adds 11 points.
- **RLHF does not measurably improve on SFT.** The +0.7 points is within noise (the standard error on 1,200 problems is about 1.4 points), and the last iterate is worse.
- **The reward model is the bottleneck.** It was trained on 22,009 checker-labelled comparisons and ranks held-out pairs correctly 57% of the time. Judging whether `(7 + 5) * 3 - 4 = 34` is right is as hard for it as for the policy.
- **Reward hacking with a looser leash.** An earlier configuration, with a weaker reward model and a looser KL penalty, saw the reward model's score rise while checked accuracy fell from 41% to 8%. The run above uses the settings that stayed stable: learning rate 5e-6, KL coefficient 0.1, 2 PPO epochs.
- **Selection on validation did not transfer.** The iterate kept looked 4.6 points better than SFT on the val split, but it was the best of 16 noisy evaluations and the gain was not there on test.
- **PPO itself works.** On a synthetic reward (short answers) it goes from 30% to 97% in 30 iterations, and a single update raises the log-probability of correct proofs relative to wrong ones. Optimising the checker

directly was no better than the learned reward: at 0.8M parameters, RL mostly redistributes probability among answers the SFT model already produces.

4.2 Replication (running)

The replication fixes the first run's weak points: * three seeds per arm, and the last iterate always (no selection on val); * 2,000 test problems instead of 1,200; * every answer compiled with Lean 4; * paired statistics; * two more arms: rejection-sampling fine-tuning, and "more SFT" on fresh references to check whether SFT was simply undertrained.

It also fixes a confound found in review. Batched generation gave every prompt the token budget of the longest prompt in its batch, which capped PPO's answers at about 105 tokens while the other arms got the full budget.

The problem generators were rebuilt for this run, so absolute accuracies are not comparable with section 4.1.

stage	result so far
pretraining (5,000 steps)	validation loss 0.537 → 0.323 (train 0.315)
SFT (3,000 steps), val split, 75 problems per domain	prop 25.3%, fol 80.0%, nat 18.7%, linear 4.0%, **all 32.0%**
SFT samples used for the reward model (48,000 at temperature 1)	checked correct: prop 21.0%, fol 71.2%, nat 13.3%, linear 10.9%
reward model: 25,622 training comparisons, 1,161 held out (251 prompts)	held-out pairwise accuracy **55.8%** (prop 58.1, fol 64.2, nat 53.4, linear 52.7); AUC for correctness 0.62
reward model fit	train loss 0.32 vs held-out loss 1.02, held-out loss rising from step 500: it memorises its comparisons
RLHF, PPO vs checker, RFT, more SFT (× 3 seeds)	running
test evaluation (2,000 problems, Lean-compiled) and pass@k	pending

The replication's reward model again sits barely above chance on prompts it has not seen. That accuracy is what PPO has to work with in the RLHF arm, so the first run's conclusion that RLHF is limited by its reward model is on track to replicate. The checker and RFT arms do not depend on the reward model.

5. What I learned

1. **RLHF is only as good as its reward model, and a reward model can be confidently wrong.** With noise-free labels and 26k comparisons, a reward model the size of the policy still did not learn to judge proofs: it fit its training pairs and stayed near chance on new prompts. PPO then had little to optimise, and when the KL leash was loosened it optimised the errors instead.
2. **Measure the measurement.** Held-out pairwise accuracy of the reward model, and reward against true accuracy during PPO, explained the result before the test numbers did. A training curve that only shows the reward would have shown the hacking run "improving".
3. **Small gains need paired statistics and seeds.** A 4.6-point validation gain was selection noise, and a 0.7-point test gain was inside the standard error. The replication therefore fixes the test set, uses paired tests and three seeds, and drops selection on val.
4. **Contamination hides in respellings.** Hashing raw prompts let about a fifth of arithmetic and a seventh of linear-equation test problems have a commuted or side-swapped twin in training. Splitting on a canonical form closes that.
5. **Check the checker.** Fuzzing the checkers against real Lean found inputs they accepted and Lean rejected: non-space whitespace, and headers deep enough to exhaust Lean's recursion limit. An RL policy optimising a checker could find such holes, so the replication compiles every evaluated answer with Lean.

Reproducing

Code, tests and the experiment script: https://github.com/rbahl1/nanogpt. `./run_experiment.sh` runs the whole replication on a 4-core CPU in a few hours, and every long stage resumes exactly after an interruption.